



PHP UNIT II javascript - notes

Bachelor's of computer Application (BCA) (University of Calicut)



Scan to open on Studocu

UNIT- II

JAVASCRIPT

JavaScript is a lightweight, interpreted programming language. It is designed for creating network-centric applications. It is complimentary to and integrated with Java. JavaScript is very easy to implement because it is integrated with HTML. It is open and cross-platform. JavaScript is a dynamic computer programming language. It is lightweight and most commonly used as a part of web pages, whose implementations allow client-side script to interact with the user and make dynamic pages. It is an interpreted programming language with object-oriented capabilities.

CLIENT SIDE PROGRAMMING

Client-side JavaScript is the most common form of the language. The script should be included in or referenced by an HTML document for the code to be interpreted by the browser.

It means that a web page need not be a static HTML, but can include programs that interact with the user, control the browser, and dynamically create HTML content.

The JavaScript client-side mechanism provides many advantages over traditional CGI server-side scripts. For example, you might use JavaScript to check if the user has entered a valid e-mail address in a form field.

The JavaScript code is executed when the user submits the form, and only if all the entries are valid, they would be submitted to the Web Server.

JavaScript can be used to trap user-initiated events such as button clicks, link navigation, and other actions that the user initiates explicitly or implicitly.

ADVANTAGES OF JAVASCRIPT

The merits of using JavaScript are –

- **Less server interaction** – You can validate user input before sending the page off to the server. This saves server traffic, which means less load on your server.
- **Immediate feedback to the visitors** – They don't have to wait for a page reload to see if they have forgotten to enter something.

- **Increased interactivity** – You can create interfaces that react when the user hovers over them with a mouse or activates them via the keyboard.
- **Richer interfaces** – You can use JavaScript to include such items as drag-and-drop components and sliders to give a Rich Interface to your site visitors.

LIMITATIONS OF JAVASCRIPT

We cannot treat JavaScript as a full-fledged programming language. It lacks the following important features –

- Client-side JavaScript does not allow the reading or writing of files. This has been kept for security reason.
- JavaScript cannot be used for networking applications because there is no such support available.
- JavaScript doesn't have any multi-threading or multiprocessor capabilities.

JAVASCRIPT – SYNTAX

SCRIPT TAG

JavaScript can be implemented using JavaScript statements that are placed within the `<script>... </script>` HTML tags in a web page.

You can place the `<script>` tags, containing your JavaScript, anywhere within your web page, but it is normally recommended that you should keep it within the `<head>` tags.

The `<script>` tag alerts the browser program to start interpreting all the text between these tags as a script. A simple syntax of your JavaScript will appear as follows.

```
<script ...>  
    JavaScript code  
</script>
```

The script tag takes two important attributes –

Language – This attribute specifies what scripting language you are using. Typically, its value will be javascript. Although recent versions of HTML (and XHTML, its successor) have phased out the use of this attribute.

Type – This attribute is what is now recommended to indicate the scripting language in use and its value should be set to "text/javascript".

So your JavaScript segment will look like –

```
<script language = "javascript" type = "text/javascript">  
    JavaScript code  
</script>
```

JAVASCRIPT EXAMPLE

Let us take a sample example to print out "Hello World". We added an optional HTML comment that surrounds our JavaScript code. This is to save our code from a browser that does not support JavaScript. The comment ends with a "`//-->`". Here "`/*`" signifies a comment in JavaScript, so we add that to prevent a browser from reading the end of the HTML comment as a piece of JavaScript code. Next, we call a function **document.write** which writes a string into our HTML document.

This function can be used to write text, HTML, or both. Take a look at the following code.

```
<html>  
  <body>  
    <script language = "javascript" type = "text/javascript">  
      <!--  
        document.write("Hello World!")  
      //-->  
    </script>  
  </body>  
</html>
```

This code will produce the following result –

Hello World!

WHITESPACE AND LINE BREAKS

JavaScript ignores spaces, tabs, and newlines that appear in JavaScript programs. You can use spaces, tabs, and newlines freely in your program and you are free to format and indent your programs in a neat and consistent way that makes the code easy to read and understand.

Semicolons are Optional

Simple statements in JavaScript are generally followed by a semicolon character, just as they are in C, C++, and Java. JavaScript, however, allows you to omit this semicolon if each of your statements are placed on a separate line. For example, the following code could be written without semicolons.

```
<script language = "javascript" type = "text/javascript">

  <!--

    var1 = 10

    var2 = 20

  //-->

</script>
```

But when formatted in a single line as follows, you must use semicolons –

```
<script language = "javascript" type = "text/javascript">

  <!--

    var1 = 10; var2 = 20;

  //-->

</script>
```

Note – It is a good programming practice to use semicolons.

Case Sensitivity

JavaScript is a case-sensitive language. This means that the language keywords, variables, function names, and any other identifiers must always be typed with a consistent capitalization of letters. So the identifiers **Time** and **TIME** will convey different meanings in JavaScript.

COMMENTS IN JAVASCRIPT

JavaScript supports both C-style and C++-style comments, Thus –

- Any text between a // and the end of a line is treated as a comment and is ignored by JavaScript.
- Any text between the characters /* and */ is treated as a comment. This may span multiple lines.
- JavaScript also recognizes the HTML comment opening sequence <!--. JavaScript treats this as a single-line comment, just as it does the // comment.
- The HTML comment closing sequence --> is not recognized by JavaScript so it should be written as //-->.

Example

The following example shows how to use comments in JavaScript.

```
<script language = "javascript" type = "text/javascript">
  <!--
    // This is a comment. It is similar to comments in C++

    /*
    * This is a multi-line comment in JavaScript
    * It is very similar to comments in C Programming
    */

    //-->
</script>
```

<NOSCRIPT> TAGS

If you have to do something important using JavaScript, then you can display a warning message to the user using **<noscript>** tags.

You can add a **noscript** block immediately after the script block as follows –

```
<html>
  <body>
```

```

<script language = "javascript" type = "text/javascript">
    <!--
        document.write("Hello World!")
    //-->
</script>

<noscript>
    Sorry...JavaScript is needed to go ahead.
</noscript>
</body>
</html>

```

Now, if the user's browser does not support JavaScript or JavaScript is not enabled, then the message from </noscript> will be displayed on the screen.

JAVASCRIPT – INCLUDING JAVASCRIPT IN HTML

There is a flexibility given to include JavaScript code anywhere in an HTML document. However the most preferred ways to include JavaScript in an HTML file are as follows –

- *Script in <head>...</head> section.*
- *Script in <body>...</body> section.*
- *Script in <body>...</body> and <head>...</head> sections.*
- *Script in an external file and then include in <head>...</head> section.*

JavaScript in <head>...</head> section

If you want to have a script run on some event, such as when a user clicks somewhere, then you will place that script in the head as follows –

```

<html>
<head>
    <script type = "text/javascript">

        <!--

            function sayHello() {

```

```

        alert("Hello World")
    }
    //-->
</script>
</head>

<body>
    <input type = "button" onclick = "sayHello()" value = "Say Hello" />
</body>
</html>

```

JavaScript in <body>...</body> section

If you need a script to run as the page loads so that the script generates content in the page, then the script goes in the <body> portion of the document. In this case, you would not have any function defined using JavaScript. Take a look at the following code.

```

<html>
<head>
</head>

<body>
    <script type = "text/javascript">
        <!--
            document.write("Hello World")
        //-->
    </script>

    <p>This is web page body </p>
</body>
</html>

```

JavaScript in <body> and <head> Sections

You can put your JavaScript code in <head> and <body> section altogether as follows –


```
<html>
<head>
  <script type = "text/javascript">
    <!--
      function sayHello() {
        alert("Hello World")
      }
    //-->
  </script>
</head>

<body>
  <script type = "text/javascript">
    <!--
      document.write("Hello World")
    //-->
  </script>

  <input type = "button" onclick = "sayHello()" value = "Say Hello" />
</body>
</html>
```

JavaScript in External File

As you begin to work more extensively with JavaScript, you will be likely to find that there are cases where you are reusing identical JavaScript code on multiple pages of a site.

You are not restricted to be maintaining identical code in multiple HTML files. **The script tag provides a mechanism to allow you to store JavaScript in an external file and then include it into your HTML files.**

Here is an example to show how you can include an external JavaScript file in your HTML code using **script** tag and its **src** attribute.

```
<html>
```

```
<head>
  <script type = "text/javascript" src = "filename.js" ></script>
</head>

<body>
  .....
</body>
</html>
```

To use JavaScript from an external file source, you need to write all your JavaScript source code in a simple text file with the extension ".js" and then include that file as shown above.

For example, you can keep the following content in **filename.js** file and then you can use **sayHello** function in your HTML file after including the filename.js file.

```
function sayHello() {
  alert("Hello World")
}
```

JAVASCRIPT DATATYPES

One of the most fundamental characteristics of a programming language is the set of data types it supports. These are the type of values that can be represented and manipulated in a programming language.

JavaScript allows you to work with three primitive data types –

- **Numbers, eg. 123, 120.50 etc.**
- **Strings of text e.g. "This text string" etc.**
- **Boolean e.g. true or false.**

JavaScript also defines two trivial data types, **null and undefined**, each of which defines only a single value. In addition to these primitive data types, JavaScript supports a composite data type known as **object**.

JAVASCRIPT VARIABLES

Like many other programming languages, JavaScript has variables. Variables can be thought of as named containers. You can place data into these containers and then refer to the data simply by naming the container.

Before you use a variable in a JavaScript program, you must declare it. Variables are declared with the **var** keyword as follows.

```
<script type = "text/javascript">
  <!--
    var money;
    var name;
  //-->
</script>
```

You can also declare multiple variables with the same **var** keyword as follows –

```
<script type = "text/javascript">
  <!--
    var money, name;
  //-->
</script>
```

Storing a value in a variable is called **variable initialization**. You can do variable initialization at the time of variable creation or at a later point in time when you need that variable.

For instance, you might create a variable named **money** and assign the value 2000.50 to it later. For another variable, you can assign a value at the time of initialization as follows.

```
<script type = "text/javascript">
  <!--
    var name = "Ali";

    var money;

    money = 2000.50;
  //-->
</script>
```

Note – Use the **var** keyword only for declaration or initialization, once for the life of any variable name in a document. You should not re-declare same variable twice.

JavaScript is **untyped** language. This means that a JavaScript variable can hold a value of any data type. Unlike many other languages, you don't have to tell JavaScript during variable

declaration what type of value the variable will hold. The value type of a variable can change during the execution of a program and JavaScript takes care of it automatically.

JavaScript Variable Scope

The scope of a variable is the region of your program in which it is defined. JavaScript variables have only two scopes.

- **Global Variables** – A global variable has global scope which means it can be defined anywhere in your JavaScript code.
- **Local Variables** – A local variable will be visible only within a function where it is defined. Function parameters are always local to that function.

Within the body of a function, a local variable takes precedence over a global variable with the same name. If you declare a local variable or function parameter with the same name as a global variable, you effectively hide the global variable. Take a look into the following example.

```
<html>
<body onload = checkscope();>
  <script type = "text/javascript">
    <!--
      var myVar = "global";    // Declare a global variable
      function checkscope() {
        var myVar = "local";  // Declare a local variable
        document.write(myVar);
      }
    //-->
  </script>
</body>
</html>
```

This produces the following result –

local

JavaScript Variable Names

While naming your variables in JavaScript, keep the following rules in mind.

- You should not use any of the JavaScript reserved keywords as a variable name. These keywords are mentioned in the next section. For example, **break** or **boolean** variable names are not valid.
- JavaScript variable names should not start with a numeral (0-9). They must begin with a letter or an underscore character. For example, **123test** is an invalid variable name but **_123test** is a valid one.
- JavaScript variable names are case-sensitive. For example, **Name** and **name** are two different variables.



JAVASCRIPT – OPERATORS

What is an Operator?

Let us take a simple expression **4 + 5 is equal to 9**. Here 4 and 5 are called **operands** and ‘+’ is called the **operator**. JavaScript supports the following types of operators.

- Arithmetic Operators
- Comparison Operators
- Logical (or Relational) Operators
- Assignment Operators
- Conditional (or ternary) Operators

Arithmetic Operators

JavaScript supports the following arithmetic operators –

Assume variable A holds 10 and variable B holds 20, then –

Sr.No.	Operator & Description
1	+ (Addition) Adds two operands Ex: A + B will give 30
2	- (Subtraction) Subtracts the second operand from the first Ex: A - B will give -10

3	* (Multiplication) Multiply both operands Ex: A * B will give 200
4	/ (Division) Divide the numerator by the denominator Ex: B / A will give 2
5	% (Modulus) Outputs the remainder of an integer division Ex: B % A will give 0
6	++ (Increment) Increases an integer value by one Ex: A++ will give 11
7	-- (Decrement) Decreases an integer value by one Ex: A-- will give 9

Note – Addition operator (+) works for Numeric as well as Strings. e.g. "a" + 10 will give "a10".

Example

The following code shows how to use arithmetic operators in JavaScript.

```
<html>
<body>

  <script type = "text/javascript">
    <!--
      var a = 33;
      var b = 10;
      var c = "Test";
      var linebreak = "<br />";

      document.write("a + b = ");
      result = a + b;
      document.write(result);
      document.write(linebreak);

      document.write("a - b = ");
      result = a - b;
```

```
document.write(result);
document.write(linebreak);
```

```
document.write("a / b = ");
result = a / b;
document.write(result);
document.write(linebreak);
```

```
document.write("a % b = ");
result = a % b;
document.write(result);
document.write(linebreak);
```

```
document.write("a + b + c = ");
result = a + b + c;
document.write(result);
document.write(linebreak);
```

```
a = ++a;
document.write("++a = ");
result = ++a;
document.write(result);
document.write(linebreak);
```

```
b = --b;
document.write("--b = ");
result = --b;
document.write(result);
document.write(linebreak);
```

```
//-->
</script>
```

Set the variables to different values and then try...

```
</body>
</html>
```

Output

```
a + b = 43
a - b = 23
a / b = 3.3
a % b = 3
a + b + c = 43Test
++a = 35
--b = 8
Set the variables to different values and then try...
```

Comparison Operators

JavaScript supports the following comparison operators –

Assume variable A holds 10 and variable B holds 20, then –

Sr.No.	Operator & Description
1	== (Equal) Checks if the value of two operands are equal or not, if yes, then the condition becomes true. Ex: (A == B) is not true.
2	!= (Not Equal) Checks if the value of two operands are equal or not, if the values are not equal, then the condition becomes true. Ex: (A != B) is true.
3	> (Greater than) Checks if the value of the left operand is greater than the value of the right operand, if yes, then the condition becomes true. Ex: (A > B) is not true.
4	< (Less than) Checks if the value of the left operand is less than the value of the right operand, if yes, then the condition becomes true. Ex: (A < B) is true.
5	>= (Greater than or Equal to) Checks if the value of the left operand is greater than or equal to the value of the right operand, if yes, then the condition becomes true. Ex: (A >= B) is not true.
6	<= (Less than or Equal to) Checks if the value of the left operand is less than or equal to the value of the right operand, if yes, then the condition becomes true. Ex: (A <= B) is true.

Example

The following code shows how to use comparison operators in JavaScript.

```
<html>
```



```
<body>
  <script type = "text/javascript">
```

```
    <!--
      var a = 10;
      var b = 20;
      var linebreak = "<br />";

      document.write("(a == b) => ");
      result = (a == b);
      document.write(result);
      document.write(linebreak);

      document.write("(a < b) => ");
      result = (a < b);
      document.write(result);
      document.write(linebreak);

      document.write("(a > b) => ");
      result = (a > b);
      document.write(result);
      document.write(linebreak);

      document.write("(a != b) => ");
      result = (a != b);
      document.write(result);
      document.write(linebreak);
      document.write("(a >= b) => ");
      result = (a >= b);
      document.write(result);
      document.write(linebreak);
      document.write("(a <= b) => ");
      result = (a <= b);
      document.write(result);
      document.write(linebreak);
    //-->
```

```
</script>
```

Set the variables to different values and different operators and then try...

```
</body>
```

```
</html>
```

Output

```
(a == b) => false
(a < b) => true
(a > b) => false
```

```
(a != b) => true
(a >= b) => false
```

```
a <= b) => true
```

Set the variables to different values and different operators and then try...

Logical Operators

JavaScript supports the following logical operators –

Assume variable A holds 10 and variable B holds 20, then –

Sr.No.	Operator & Description
1	&& (Logical AND) If both the operands are non-zero, then the condition becomes true. Ex: (A && B) is true.
2	 (Logical OR) If any of the two operands are non-zero, then the condition becomes true. Ex: (A B) is true.
3	! (Logical NOT) Reverses the logical state of its operand. If a condition is true, then the Logical NOT operator will make it false. Ex: ! (A && B) is false.

Example

Try the following code to learn how to implement Logical Operators in JavaScript.

```
<html>
<body>
  <script type = "text/javascript">
    <!--
      var a = true;
      var b = false;
      var linebreak = "<br />";

      document.write("(a && b) => ");
      result = (a && b);
      document.write(result);
      document.write(linebreak);
```

```

document.write("(a || b) => ");
result = (a || b);
document.write(result);
document.write(linebreak);

document.write("!(a && b) => ");
result = (!(a && b));
document.write(result);
document.write(linebreak);

//-->
</script>
<p>Set the variables to different values and different operators and then try...</p>
</body>
</html>

```

Output

```

(a && b) => false
(a || b) => true
!(a && b) => true
Set the variables to different values and different operators and then try...

```

Bitwise Operators

JavaScript supports the following bitwise operators –

Assume variable A holds 2 and variable B holds 3, then –

Sr.No.	Operator & Description
1	& (Bitwise AND) It performs a Boolean AND operation on each bit of its integer arguments. Ex: (A & B) is 2.
2	 (BitWise OR) It performs a Boolean OR operation on each bit of its integer arguments. Ex: (A B) is 3.

3	^ (Bitwise XOR) It performs a Boolean exclusive OR operation on each bit of its integer arguments. Exclusive OR means that either operand one is true or operand two is true, but not both. Ex: (A ^ B) is 1.
4	~ (Bitwise Not) It is a unary operator and operates by reversing all the bits in the operand. Ex: (~B) is -4.
5	<< (Left Shift) It moves all the bits in its first operand to the left by the number of places specified in the second operand. New bits are filled with zeros. Shifting a value left by one position is equivalent to multiplying it by 2, shifting two positions is equivalent to multiplying by 4, and so on. Ex: (A << 1) is 4.
6	>> (Right Shift) Binary Right Shift Operator. The left operand's value is moved right by the number of bits specified by the right operand. Ex: (A >> 1) is 1.
7	>>> (Right shift with Zero) This operator is just like the >> operator, except that the bits shifted in on the left are always zero. Ex: (A >>> 1) is 1.

Example

Try the following code to implement Bitwise operator in JavaScript.

```
<html>
<body>
  <script type = "text/javascript">
    <!--
      var a = 2; // Bit presentation 10
      var b = 3; // Bit presentation 11
      var linebreak = "<br />";

      document.write("(a & b) => ");
```

```

result = (a & b);
document.write(result);
document.write(linebreak);

document.write("(a | b) ==> ");
result = (a | b);
document.write(result);
document.write(linebreak);

document.write("(a ^ b) ==> ");
result = (a ^ b);
document.write(result);
document.write(linebreak);

document.write("(~b) ==> ");
result = (~b);
document.write(result);
document.write(linebreak);

document.write("(a << b) ==> ");
result = (a << b);
document.write(result);
document.write(linebreak);

document.write("(a >> b) ==> ");
result = (a >> b);
document.write(result);
document.write(linebreak);
//-->
</script>
<p>Set the variables to different values and different operators and then try...</p>
</body>
</html>

```

```

(a & b) ==> 2
(a | b) ==> 3
(a ^ b) ==> 1
(~b) ==> -4
(a << b) ==> 16
(a >> b) ==> 0

```

Set the variables to different values and different operators and then try...

Assignment Operators

JavaScript supports the following assignment operators –

Sr.No.	Operator & Description
1	= (Simple Assignment) Assigns values from the right side operand to the left side operand Ex: C = A + B will assign the value of A + B into C
2	+= (Add and Assignment) It adds the right operand to the left operand and assigns the result to the left operand. Ex: C += A is equivalent to C = C + A
3	-= (Subtract and Assignment) It subtracts the right operand from the left operand and assigns the result to the left operand. Ex: C -= A is equivalent to C = C - A
4	*= (Multiply and Assignment) It multiplies the right operand with the left operand and assigns the result to the left operand. Ex: C *= A is equivalent to C = C * A
5	/= (Divide and Assignment) It divides the left operand with the right operand and assigns the result to the left operand. Ex: C /= A is equivalent to C = C / A
6	%= (Modules and Assignment) It takes modulus using two operands and assigns the result to the left operand. Ex: C %= A is equivalent to C = C % A

Note – Same logic applies to Bitwise operators so they will become like <<=, >>=, >>=, &=, |= and ^=.

Example

Try the following code to implement assignment operator in JavaScript.

```
<html>
<body>
  <script type = "text/javascript">
    <!--
      var a = 33;
```

```
var b = 10;
var linebreak = "<br />";

document.write("Value of a => (a = b) => ");
result = (a = b);
document.write(result);
document.write(linebreak);

document.write("Value of a => (a += b) => ");
result = (a += b);
document.write(result);
document.write(linebreak);

document.write("Value of a => (a -= b) => ");
result = (a -= b);
document.write(result);
document.write(linebreak);

document.write("Value of a => (a *= b) => ");
result = (a *= b);
document.write(result);
document.write(linebreak);

document.write("Value of a => (a /= b) => ");
result = (a /= b);
document.write(result);
document.write(linebreak);

document.write("Value of a => (a %= b) => ");
result = (a %= b);
document.write(result);
document.write(linebreak);
```

```
//-->
```

```
</script>
```

```

<p>Set the variables to different values and different operators and then try...</p>
</body>
</html>

```

Output

```

Value of a => (a = b) => 10
Value of a => (a += b) => 20
Value of a => (a -= b) => 10
Value of a => (a *= b) => 100
Value of a => (a /= b) => 10
Value of a => (a %= b) => 0
Set the variables to different values and different operators and then try...

```

Miscellaneous Operator

We will discuss two operators here that are quite useful in JavaScript: the **conditional operator** (?:) and the **typeof operator**.

Conditional Operator (?:)

The conditional operator first evaluates an expression for a true or false value and then executes one of the two given statements depending upon the result of the evaluation.

Sr.No.	Operator and Description
1	? : (Conditional) If Condition is true? Then value X : Otherwise value Y

Example

Try the following code to understand how the Conditional Operator works in JavaScript.

```

<html>
<body>
  <script type = "text/javascript">
    <!--
      var a = 10;
      var b = 20;
      var linebreak = "<br />";

      document.write ("((a > b) ? 100 : 200) => ");
      result = (a > b) ? 100 : 200;
      document.write(result);
      document.write(linebreak);

      document.write ("((a < b) ? 100 : 200) => ");
    </script>
  </body>
</html>

```



```

    result = (a < b) ? 100 : 200;
    document.write(result);
    document.write(linebreak);
    //-->
</script>
<p>Set the variables to different values and different operators and then try...</p>
</body>
</html>

```

Output

```

((a > b) ? 100 : 200) => 200
((a < b) ? 100 : 200) => 100
Set the variables to different values and different operators and then try...

```

typeof Operator

The **typeof** operator is a unary operator that is placed before its single operand, which can be of any type. Its value is a string indicating the data type of the operand.

The *typeof* operator evaluates to "number", "string", or "boolean" if its operand is a number, string, or boolean value and returns true or false based on the evaluation.

Here is a list of the return values for the **typeof** Operator.

Type	String Returned by typeof
Number	"number"
String	"string"
Boolean	"boolean"
Object	"object"
Function	"function"
Undefined	"undefined"
Null	"object"

Example

The following code shows how to implement **typeof** operator.

```

<html>
<body>

```

```

<script type = "text/javascript">
  <!--
    var a = 10;
    var b = "String";
    var linebreak = "<br />";

    result = (typeof b == "string" ? "B is String" : "B is Numeric");
    document.write("Result => ");
    document.write(result);
    document.write(linebreak);

    result = (typeof a == "string" ? "A is String" : "A is Numeric");
    document.write("Result => ");
    document.write(result);
    document.write(linebreak);
  //-->
</script>
<p>Set the variables to different values and different operators and then try...</p>
</body>
</html>

```

Output

```

Result => B is String
Result => A is Numeric
Set the variables to different values and different operators and then try...

```

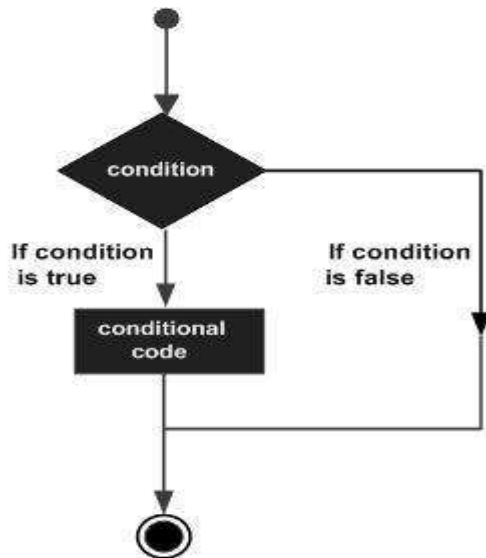
JAVASCRIPT - IF...ELSE STATEMENT

While writing a program, there may be a situation when you need to adopt one out of a given set of paths. In such cases, you need to use conditional statements that allow your program to make correct decisions and perform right actions.

JavaScript supports conditional statements which are used to perform different actions based on different conditions. Here we will explain the **if..else** statement.

Flow Chart of if-else

The following flow chart shows how the if-else statement works.



JavaScript supports the following forms of **if..else** statement –

- if statement
- if...else statement
- if...else if... statement.

if statement

The **if** statement is the fundamental control statement that allows JavaScript to make decisions and execute statements conditionally.

Syntax

The syntax for a basic if statement is as follows –

```

if (expression) {
    Statement(s) to be executed if expression is true
}
  
```

Here a JavaScript expression is evaluated. If the resulting value is true, the given statement(s) are executed. If the expression is false, then no statement would be not executed. Most of the times, you will use comparison operators while making decisions.

Example

Try the following example to understand how the **if** statement works.

```

<html>
<body>
  <script type = "text/javascript">
    <!--
      var age = 20;
  </script>
</body>
</html>
  
```

```

    if( age > 18 ) {
        document.write("<b>Qualifies for driving</b>");
    }
    <!-->
</script>
<p>Set the variable to different value and then try...</p>
</body>
</html>

```

Output

Qualifies for driving
Set the variable to different value and then try...

if...else statement

The 'if...else' statement is the next form of control statement that allows JavaScript to execute statements in a more controlled way.

Syntax

```

if (expression) {
    Statement(s) to be executed if expression is true
} else {
    Statement(s) to be executed if expression is false
}

```

Here JavaScript expression is evaluated. If the resulting value is true, the given statement(s) in the 'if' block, are executed. If the expression is false, then the given statement(s) in the else block are executed.

Example

Try the following code to learn how to implement an if-else statement in JavaScript.

```

<html>
<body>
    <script type = "text/javascript">
        <!--
            var age = 15;

            if( age > 18 ) {
                document.write("<b>Qualifies for driving</b>");
            } else {
                document.write("<b>Does not qualify for driving</b>");
            }
        <!-->
    </script>
    <p>Set the variable to different value and then try...</p>
</body>
</html>

```

Output

Does not qualify for driving

Set the variable to different value and then try...

if...else if... statement

The **if...else if...** statement is an advanced form of **if...else** that allows JavaScript to make a correct decision out of several conditions.

Syntax

The syntax of an if-else-if statement is as follows –

```
if (expression 1) {
    Statement(s) to be executed if expression 1 is true
} else if (expression 2) {
    Statement(s) to be executed if expression 2 is true
} else if (expression 3) {
    Statement(s) to be executed if expression 3 is true
} else {
    Statement(s) to be executed if no expression is true
}
```

There is nothing special about this code. It is just a series of **if** statements, where each **if** is a part of the **else** clause of the previous statement. Statement(s) are executed based on the true condition, if none of the conditions is true, then the **else** block is executed.

Example

Try the following code to learn how to implement an if-else-if statement in JavaScript.

```
<html>
<body>
  <script type = "text/javascript">
    <!--
      var book = "maths";
      if( book == "history" ) {
        document.write("<b>History Book</b>");
      } else if( book == "maths" ) {
        document.write("<b>Maths Book</b>");
      } else if( book == "economics" ) {
        document.write("<b>Economics Book</b>");
      } else {
        document.write("<b>Unknown Book</b>");
      }
    <!-->
  </script>
  <p>Set the variable to different value and then try...</p>
</body>
</html>
```

Output

Maths Book

Set the variable to different value and then try...

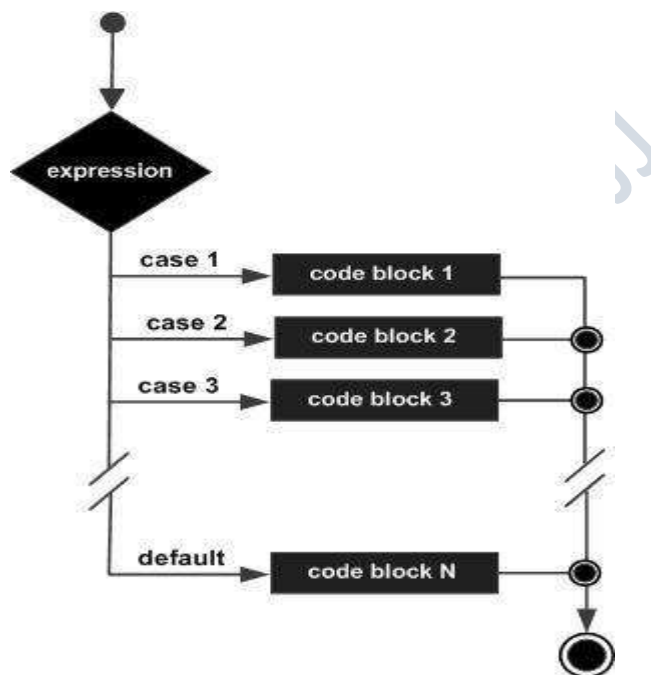
JAVASCRIPT - SWITCH CASE

You can use multiple **if...else...if** statements, as in the previous chapter, to perform a multiway branch. However, this is not always the best solution, especially when all of the branches depend on the value of a single variable.

Starting with JavaScript 1.2, you can use a **switch** statement which handles exactly this situation, and it does so more efficiently than repeated **if...else if** statements.

Flow Chart

The following flow chart explains a switch-case statement works.



Syntax

The objective of a **switch** statement is to give an expression to evaluate and several different statements to execute based on the value of the expression. The interpreter checks each **case** against the value of the expression until a match is found. If nothing matches, a **default** condition will be used.

```
switch (expression) {
  case condition 1: statement(s)
```

```

break;

case condition 2: statement(s)
break;
...

case condition n: statement(s)
break;

default: statement(s)
}

```

The **break** statements indicate the end of a particular case. If they were omitted, the interpreter would continue executing each statement in each of the following cases.

We will explain **break** statement in **Loop Control** chapter.

Example

Try the following example to implement switch-case statement.

```

<html>
<body>
  <script type = "text/javascript">
    <!--
      var grade = 'A';
      document.write("Entering switch block<br />");
      switch (grade) {
        case 'A': document.write("Good job<br />");
          break;

        case 'B': document.write("Pretty good<br />");
          break;

        case 'C': document.write("Passed<br />");
          break;

        case 'D': document.write("Not so good<br />");
          break;

        case 'F': document.write("Failed<br />");
          break;

        default: document.write("Unknown grade<br />")
      }
      document.write("Exiting switch block");
    //-->
  </script>
  <p>Set the variable to different value and then try...</p>
</body>
</html>

```

Output

Entering switch block
Good job
Exiting switch block
Set the variable to different value and then try...

Break statements play a major role in switch-case statements. Try the following code that uses switch-case statement without any break statement.

```
<html>

<body>

  <script type = "text/javascript">

    <!--

      var grade = 'A';

      document.write("Entering switch block<br />");

      switch (grade) {

        case 'A': document.write("Good job<br />");

        case 'B': document.write("Pretty good<br />");

        case 'C': document.write("Passed<br />");

        case 'D': document.write("Not so good<br />");

        case 'F': document.write("Failed<br />");

        default: document.write("Unknown grade<br />")

      }

      document.write("Exiting switch block");

    //-->

  </script>

  <p>Set the variable to different value and then try...</p>

</body>

</html>
```

Output

Entering switch block
Good job

Pretty good
 Passed
 Not so good
 Failed
 Unknown grade
 Exiting switch block
 Set the variable to different value and then try..

JAVASCRIPT - WHILE LOOPS

While writing a program, you may encounter a situation where you need to perform an action over and over again. In such situations, you would need to write loop statements to reduce the number of lines.

JavaScript supports all the necessary loops to ease down the pressure of programming.

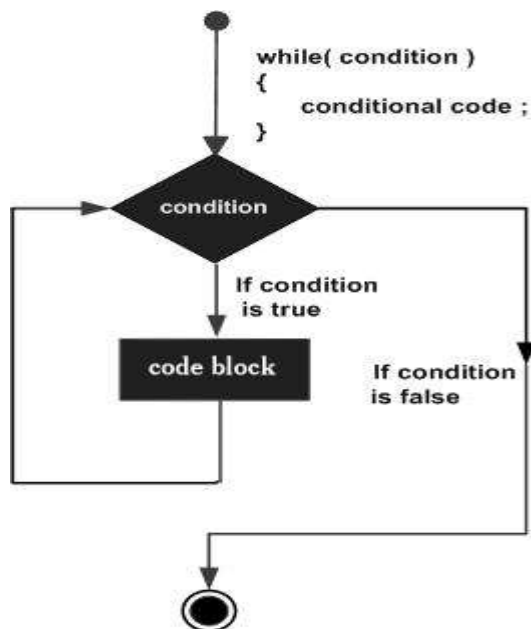
The while Loop

The most basic loop in JavaScript is the **while** loop which would be discussed in this chapter.

The purpose of a **while** loop is to execute a statement or code block repeatedly as long as an **expression** is true. Once the expression becomes **false**, the loop terminates.

Flow Chart

The flow chart of **while loop** looks as follows –



Syntax

The syntax of **while loop** in JavaScript is as follows –

```
while (expression) {
```

```

Statement(s) to be executed if expression is true
}

```

Example

Try the following example to implement while loop.

```

<html>
<body>

  <script type = "text/javascript">
    <!--
      var count = 0;
      document.write("Starting Loop ");

      while (count < 10) {
        document.write("Current Count : " + count + "<br />");
        count++;
      }

      document.write("Loop stopped!");
    //-->
  </script>

  <p>Set the variable to different value and then try...</p>
</body>
</html>

```

Output

```

Starting Loop
Current Count : 0
Current Count : 1
Current Count : 2
Current Count : 3
Current Count : 4
Current Count : 5
Current Count : 6
Current Count : 7
Current Count : 8
Current Count : 9
Loop stopped!
Set the variable to different value and then try...

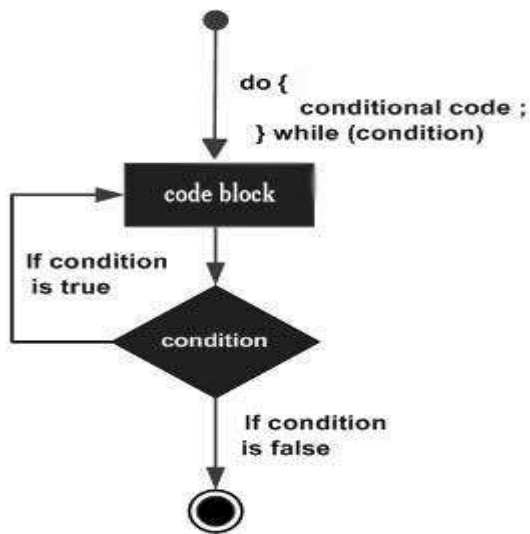
```

The do...while Loop

The **do...while** loop is similar to the **while** loop except that the condition check happens at the end of the loop. This means that the loop will always be executed at least once, even if the condition is **false**.

Flow Chart

The flow chart of a **do-while** loop would be as follows –



Syntax

The syntax for **do-while** loop in JavaScript is as follows –

```
do {
  Statement(s) to be executed;
} while (expression);
```

Note – Don't miss the semicolon used at the end of the **do...while** loop.

Example

Try the following example to learn how to implement a **do-while** loop in JavaScript.

```
<html>
<body>
  <script type = "text/javascript">
    <!--
      var count = 0;

      document.write("Starting Loop" + "<br />");
      do {
        document.write("Current Count : " + count + "<br />");
        count++;
      }

      while (count < 5);
```

```

        document.write ("Loop stopped!");

    //-->

</script>

<p>Set the variable to different value and then try...</p>

</body>

</html>

```

Output

```

Starting Loop
Current Count : 0
Current Count : 1
Current Count : 2
Current Count : 3
Current Count : 4
Loop Stopped!
Set the variable to different value and then try...

```

JAVASCRIPT - FOR LOOP

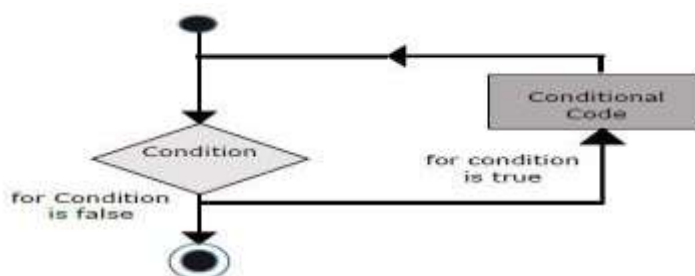
The **'for'** loop is the most compact form of looping. It includes the following three important parts –

- The **loop initialization** where we initialize our counter to a starting value. The initialization statement is executed before the loop begins.
- The **test statement** which will test if a given condition is true or not. If the condition is true, then the code given inside the loop will be executed, otherwise the control will come out of the loop.
- The **iteration statement** where you can increase or decrease your counter.

You can put all the three parts in a single line separated by semicolons.

Flow Chart

The flow chart of a **for** loop in JavaScript would be as follows –



Syntax

The syntax of **for** loop in JavaScript is as follows –

```
for (initialization; test condition; iteration statement) {  
    Statement(s) to be executed if test condition is true  
}
```

Example

Try the following example to learn how a **for** loop works in JavaScript.

```
<html>  
<body>  
    <script type = "text/javascript">  
        <!--  
            var count;  
            document.write("Starting Loop" + "<br />");  
            for(count = 0; count < 10; count++) {  
                document.write("Current Count : " + count );  
                document.write("<br />");  
            }  
            document.write("Loop stopped!");  
        //-->  
    </script>  
    <p>Set the variable to different value and then try...</p>  
</body>  
</html>
```

Output

```
Starting Loop  
Current Count : 0  
Current Count : 1  
Current Count : 2  
Current Count : 3  
Current Count : 4  
Current Count : 5  
Current Count : 6  
Current Count : 7  
Current Count : 8  
Current Count : 9  
Loop stopped!  
Set the variable to different value and then try...
```

JAVASCRIPT FOR...IN LOOP

The **for...in** loop is used to loop through an object's properties. As we have not discussed Objects yet, you may not feel comfortable with this loop. But once you understand how objects behave in JavaScript, you will find this loop very useful.

Syntax

The syntax of 'for..in' loop is –

```
for (variablename in object) {
    statement or block to execute
}
```

In each iteration, one property from **object** is assigned to **variablename** and this loop continues till all the properties of the object are exhausted.

Example

Try the following example to implement 'for-in' loop. It prints the web browser's **Navigator** object.

```
<html>
<body>
  <script type = "text/javascript">
    <!--
      var aProperty;
      document.write("Navigator Object Properties<br /> ");
      for (aProperty in navigator) {
        document.write(aProperty);
        document.write("<br />");
      }
      document.write ("Exiting from the loop!");
    //-->
  </script>
  <p>Set the variable to different object and then try...</p>
</body>
</html>
```

Output

```
Navigator Object Properties
serviceWorker
webkitPersistentStorage
```

```
webkitTemporaryStorage
geolocation
doNotTrack
onLine
languages
language
userAgent
product
platform
appVersion
appName
appCodeName
hardwareConcurrency
maxTouchPoints
vendorSub
vendor
productSub
cookieEnabled
mimeTypes
plugins
javaEnabled
getStorageUpdates
getGamepads
webkitGetUserMedia
vibrate
getBattery
sendBeacon
registerProtocolHandler
unregisterProtocolHandler
Exiting from the loop!
Set the variable to different object and then try...
```

JAVASCRIPT - LOOP CONTROL

JavaScript provides full control to handle loops and switch statements. There may be a situation when you need to come out of a loop without reaching its bottom. There may also be a situation when you want to skip a part of your code block and start the next iteration of the loop.

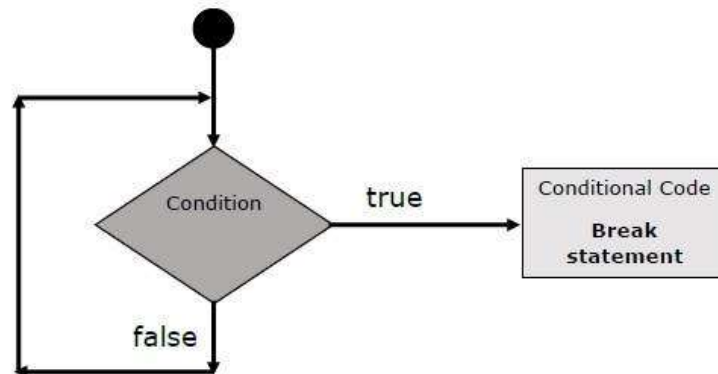
To handle all such situations, JavaScript provides **break** and **continue** statements. These statements are used to immediately come out of any loop or to start the next iteration of any loop respectively.

THE BREAK STATEMENT

The **break** statement, which was briefly introduced with the *switch* statement, is used to exit a loop early, breaking out of the enclosing curly braces.

Flow Chart

The flow chart of a break statement would look as follows –



Example

The following example illustrates the use of a **break** statement with a while loop. Notice how the loop breaks out early once **x** reaches 5 and reaches to **document.write (..)** statement just below to the closing curly brace –

```

<html>
<body>
  <script type = "text/javascript">
    <!--
    var x = 1;
    document.write("Entering the loop<br /> ");

    while (x < 20) {
      if (x == 5) {
        break; // breaks out of loop completely
      }
      x = x + 1;
      document.write( x + "<br />");
    }
    document.write("Exiting the loop!<br /> ");
    //-->
  </script>

  <p>Set the variable to different value and then try...</p>
  
```



```
</body>
</html>
```

Output

```
Entering the loop
2
3
4
5
Exiting the loop!
Set the variable to different value and then try...
```

We already have seen the usage of **break** statement inside a **switch** statement.

THE CONTINUE STATEMENT

The **continue** statement tells the interpreter to immediately start the next iteration of the loop and skip the remaining code block. When a **continue** statement is encountered, the program flow moves to the loop check expression immediately and if the condition remains true, then it starts the next iteration, otherwise the control comes out of the loop.

Example

This example illustrates the use of a **continue** statement with a while loop. Notice how the **continue** statement is used to skip printing when the index held in variable **x** reaches 5 –

```
<html>
<body>
  <script type = "text/javascript">
    <!--
      var x = 1;
      document.write("Entering the loop<br /> ");

      while (x < 10) {
        x = x + 1;

        if (x == 5) {
          continue; // skip rest of the loop body
        }
        document.write( x + "<br />");
      }
      document.write("Exiting the loop!<br /> ");
```

```

    //-->
</script>
<p>Set the variable to different value and then try...</p>
</body>
</html>

```

Output

```

Entering the loop
2
3
4
6
7
8
9
10
Exiting the loop!
Set the variable to different value and then try...

```

USING LABELS TO CONTROL THE FLOW

Starting from JavaScript 1.2, a label can be used with **break** and **continue** to control the flow more precisely. A **label** is simply an identifier followed by a colon (:) that is applied to a statement or a block of code. We will see two different examples to understand how to use labels with break and continue.

Note – Line breaks are not allowed between the ‘**continue**’ or ‘**break**’ statement and its label name. Also, there should not be any other statement in between a label name and associated loop.

Example 1

The following example shows how to implement Label with a break statement.

```

<html>
<body>
  <script type = "text/javascript">
    <!--
      document.write("Entering the loop!<br /> ");
      outerloop:    // This is the label name
      for (var i = 0; i < 5; i++) {

```

```
document.write("Outerloop: " + i + "<br />");
innerloop:
for (var j = 0; j < 5; j++) {
    if (j > 3 ) break ;           // Quit the innermost loop
    if (i == 2) break innerloop; // Do the same thing
    if (i == 4) break outerloop; // Quit the outer loop
    document.write("Innerloop: " + j + " <br />");
}
}
document.write("Exiting the loop!<br /> ");
//-->
</script>
</body> </html>
```

Output

Entering the loop!

Outerloop: 0

Innerloop: 0

Innerloop: 1

Innerloop: 2

Innerloop: 3

Outerloop: 1

Innerloop: 0

Innerloop: 1

Innerloop: 2

Innerloop: 3

Outerloop: 2

Outerloop: 3

Innerloop: 0

Innerloop: 1

Innerloop: 2

Innerloop: 3

Outerloop: 4

Exiting the loop!

JAVASCRIPT – FUNCTIONS

A function is a group of reusable code which can be called anywhere in your program. This eliminates the need of writing the same code again and again. It helps programmers in writing modular codes. Functions allow a programmer to divide a big program into a number of small and manageable functions. Like any other advanced programming language, JavaScript also supports all the features necessary to write modular code using functions. You must have seen functions like **alert()** and **write()** in the earlier chapters. We were using these functions again and again, but they had been written in core JavaScript only once.

Function Definition

Before we use a function, we need to define it. The most common way to define a function in JavaScript is by using the **function** keyword, followed by a unique function name, a list of parameters (that might be empty), and a statement block surrounded by curly braces.

Syntax

The basic syntax is shown here.

```
<script type = "text/javascript">
  <!--
    function functionname(parameter-list) {
      statements
    }
  //-->
</script>
```

Example

Try the following example. It defines a function called say Hello that takes no parameters –

```
<script type = "text/javascript">
  <!--
    function sayHello() {
      alert("Hello there");
    }
  //-->
</script>
```

Calling a Function

To invoke a function somewhere later in the script, you would simply need to write the name of that function as shown in the following code.

```
<html>
<head>
  <script type = "text/javascript">
    function sayHello() {
      document.write ("Hello there!");
    }
  </script>
</head>

<body>
  <p>Click the following button to call the function</p>
  <form>
    <input type = "button" onclick = "sayHello()" value = "Say Hello">
  </form>
  <p>Use different text in write method and then try...</p>
</body>
</html>
```

Function Parameters

Till now, we have seen functions without parameters. But there is a facility to pass different parameters while calling a function. These passed parameters can be captured inside the function and any manipulation can be done over those parameters. A function can take multiple parameters separated by comma.

Example

Try the following example. We have modified our **say Hello** function here. Now it takes two parameters.

```
<html>
<head>
  <script type = "text/javascript">
    function sayHello(name, age) {
      document.write (name + " is " + age + " years old.");
    }
  </script>
</head>
<body>
  <p>Click the following button to call the function</p>
  <form>
    <input type = "button" onclick = "sayHello('John', 25)" value = "Say Hello">
  </form>
  <p>Use different text in write method and then try...</p>
</body>
</html>
```

```
</script>

</head>

<body>
  <p>Click the following button to call the function</p>
  <form>
    <input type = "button" onclick = "sayHello('Zara', 7)" value = "Say Hello">
  </form>
  <p>Use different parameters inside the function and then try...</p>
</body>
</html>
```

The return Statement

A JavaScript function can have an optional **return** statement. This is required if you want to return a value from a function. This statement should be the last statement in a function.

For example, you can pass two numbers in a function and then you can expect the function to return their multiplication in your calling program.

Example

Try the following example. It defines a function that takes two parameters and concatenates them before returning the resultant in the calling program.

```
<html>

<head>
  <script type = "text/javascript">
    function concatenate(first, last) {
      var full;
      full = first + last;
      return full;
    }
    function secondFunction() {
      var result;
      result = concatenate('Zara', 'Ali');
```

```

        document.write (result );
    }
</script>
</head>

<body>
    <p>Click the following button to call the function</p>
    <form>
        <input type = "button" onclick = "secondFunction()" value = "Call Function">
    </form>

    <p>Use different parameters inside the function and then try...</p>
</body>
</html>

```

JAVASCRIPT BUILT-IN FUNCTIONS

Number Methods

The Number object contains only the default methods that are part of every object's definition.

Sr.No.	Method & Description
1.	toFixed() Formats a number with a specific number of digits to the right of the decimal.
2.	toLocaleString() Returns a string value version of the current number in a format that may vary according to a browser's locale settings.
3.	toPrecision() Defines how many total digits (including digits to the left and right of the decimal) to display of a number.
4.	toString() Returns the string representation of the number's value.
5.	valueOf() Returns the number's value.

Boolean Methods

Here is a list of each method and its description.

Sr.No.	Method & Description
1	<u>toSource()</u> Returns a string containing the source of the Boolean object; you can use this string to create an equivalent object.
2	<u>toString()</u> Returns a string of either "true" or "false" depending upon the value of the object.
3	<u>valueOf()</u> Returns the primitive value of the Boolean object.

String Methods

Here is a list of each method and its description.

Sr.No.	Method & Description
1.	<u>charAt()</u> Returns the character at the specified index.
2.	<u>concat()</u> Combines the text of two strings and returns a new string.
3.	<u>length()</u> Returns the length of the string.
4.	<u>match()</u> Used to match a regular expression against a string.
5.	<u>replace()</u> Used to find a match between a regular expression and a string, and to replace the matched substring with a new substring.
6.	<u>search()</u> Executes the search for a match between a regular expression and a specified string.

7.	<u>split()</u> Splits a String object into an array of strings by separating the string into substrings.
8.	<u>toLocaleLowerCase()</u> The characters within a string are converted to lower case while respecting the current locale.
9.	<u>toLocaleUpperCase()</u> The characters within a string are converted to upper case while respecting the current locale.
10.	<u>toLowerCase()</u> Returns the calling string value converted to lower case.
11.	<u>toString()</u> Returns a string representing the specified object.
12.	<u>toUpperCase()</u> Returns the calling string value converted to uppercase.
13.	<u>valueOf()</u> Returns the primitive value of the specified object.

Array Methods

Here is a list of each method and its description.

Sr.No.	Method & Description
1.	<u>concat()</u> Returns a new array comprised of this array joined with other array(s) and/or value(s).
2.	<u>indexOf()</u> Returns the first (least) index of an element within the array equal to the specified value, or -1 if none is found.
3.	<u>join()</u> Joins all elements of an array into a string.

4.	<u>pop()</u> Removes the last element from an array and returns that element.
5.	<u>push()</u> Adds one or more elements to the end of an array and returns the new length of the array.
6.	<u>reverse()</u> Reverses the order of the elements of an array -- the first becomes the last, and the last becomes the first.
7.	<u>sort()</u> Sorts the elements of an array.
8.	<u>toString()</u> Returns a string representing the array and its elements.

Date Methods

Here is a list of each method and its description.

Sr.No.	Method & Description
1.	<u>Date()</u> Returns today's date and time
2.	<u>getDate()</u> Returns the day of the month for the specified date according to local time.
3.	<u>getDay()</u> Returns the day of the week for the specified date according to local time.
4.	<u>getFullYear()</u> Returns the year of the specified date according to local time.
5.	<u>getHours()</u> Returns the hour in the specified date according to local time.
6.	<u>getMilliseconds()</u> Returns the milliseconds in the specified date according to local time.
7.	<u>getMinutes()</u>

	Returns the minutes in the specified date according to local time.
8.	<u>getMonth()</u> Returns the month in the specified date according to local time.
9.	<u>getSeconds()</u> Returns the seconds in the specified date according to local time.
10.	<u>getTime()</u> Returns the numeric value of the specified date as the number of milliseconds since January 1, 1970, 00:00:00 UTC.
11.	<u>getYear()</u> Deprecated - Returns the year in the specified date according to local time. Use <code>getFullYear</code> instead.
12.	<u>setDate()</u> Sets the day of the month for a specified date according to local time.
13.	<u>setFullYear()</u> Sets the full year for a specified date according to local time.
14.	<u>setHours()</u> Sets the hours for a specified date according to local time.
15.	<u>setMilliseconds()</u> Sets the milliseconds for a specified date according to local time.
16.	<u>setMinutes()</u> Sets the minutes for a specified date according to local time.
17.	<u>setMonth()</u> Sets the month for a specified date according to local time.
18.	<u>setSeconds()</u> Sets the seconds for a specified date according to local time.
19.	<u>setTime()</u> Sets the Date object to the time represented by a number of milliseconds since January 1, 1970, 00:00:00 UTC.
20.	<u>setYear()</u> Deprecated - Sets the year for a specified date according to local time. Use <code>setFullYear</code> instead.

Math Methods

Here is a list of each method and its description.

Sr.No.	Method & Description
21.	<u>abs()</u> Returns the absolute value of a number.
22.	<u>acos()</u> Returns the arccosine (in radians) of a number.
23.	<u>asin()</u> Returns the arcsine (in radians) of a number.
24.	<u>atan()</u> Returns the arctangent (in radians) of a number.
25.	<u>cos()</u> Returns the cosine of a number.
26.	<u>exp()</u> Returns E^N , where N is the argument, and E is Euler's constant, the base of the natural logarithm.
27.	<u>max()</u> Returns the largest of zero or more numbers.
28.	<u>min()</u> Returns the smallest of zero or more numbers.
29.	<u>pow()</u> Returns base to the exponent power, that is, base exponent.
30.	<u>sin()</u> Returns the sine of a number.
31.	<u>sqrt()</u> Returns the square root of a number.
32.	<u>tan()</u> Returns the tangent of a number.
33.	<u>toSource()</u> Returns the string "Math".

JAVASCRIPT - DIALOG BOXES (POP-UP)

JavaScript supports three important types of dialog boxes. These dialog boxes can be used to raise and alert, or to get confirmation on any input or to have a kind of input from the users. Here we will discuss each dialog box one by one.

Alert Dialog Box

An alert dialog box is mostly used to give a warning message to the users. For example, if one input field requires to enter some text but the user does not provide any input, then as a part of validation, you can use an alert box to give a warning message.

Nonetheless, an alert box can still be used for friendlier messages. Alert box gives only one button "OK" to select and proceed.

Example

```
<html>
<head>
  <script type = "text/javascript">
    <!--
      function Warn() {
        alert ("This is a warning message!");
        document.write ("This is a warning message!");
      }
    //-->
  </script>
</head>

<body>
  <p>Click the following button to see the result: </p>
  <form>
    <input type = "button" value = "Click Me" onclick = "Warn();" />
  </form>
</body>
</html>
```

Confirmation Dialog Box

A confirmation dialog box is mostly used to take user's consent on any option. It displays a dialog box with two buttons: **OK** and **Cancel**.

If the user clicks on the OK button, the window method **confirm()** will return true. If the user clicks on the Cancel button, then **confirm()** returns false. You can use a confirmation dialog box as follows.

Example

```
<html>
<head>
  <script type = "text/javascript">
    <!--
      function getConfirmation() {
        var retVal = confirm("Do you want to continue ?");
        if( retVal == true ) {
          document.write ("User wants to continue!");
          return true;
        } else {
          document.write ("User does not want to continue!");
          return false;
        }
      }
    //-->
  </script>
</head>

<body>
  <p>Click the following button to see the result: </p>
  <form>
    <input type = "button" value = "Click Me" onclick = "getConfirmation();" />
  </form>
</body>
</html>
```

Prompt Dialog Box

The prompt dialog box is very useful when you want to pop-up a text box to get user input. Thus, it enables you to interact with the user. The user needs to fill in the field and then click OK.

This dialog box is displayed using a method called **prompt()** which takes two parameters: (i) a label which you want to display in the text box and (ii) a default string to display in the text box.

This dialog box has two buttons: **OK** and **Cancel**. If the user clicks the OK button, the window method **prompt()** will return the entered value from the text box. If the user clicks the Cancel button, the window method **prompt()** returns **null**.

Example

The following example shows how to use a prompt dialog box –

```
<html>
<head>
  <script type = "text/javascript">
    <!--
      function getValue() {
        var retVal = prompt("Enter your name : ", "your name here");
        document.write("You have entered : " + retVal);
      }
    //-->
  </script>
</head>

<body>
  <p>Click the following button to see the result: </p>
  <form>
    <input type = "button" value = "Click Me" onclick = "getValue();" />
  </form>
</body>
</html>
```

isNaN() Method

The Number.isNaN() method determines whether a value is NaN (Not-A-Number).

This method returns true if the value is of the type Number, and equates to NaN. Otherwise it returns false.

Examples:

Input : 0/0

Output : true

Input : 213

Output : false

Input : '213'

Output : false

Input : 'hello'

Output : false

Input : NaN

Output : true

parseInt()

The parseInt() is an inbuilt function in JavaScript which is used to accept the string and convert it into an integer. It actually returns an integer of the specified radix(base) which is accepted as second parameter of the parseInt() function. If the string does not contain a numeric value then it returns NaN i.e, not a number.

Syntax:

parseInt(Value, radix)

Parameters:

Value: "Value" contains a string which is converted to an integer.

radix: This parameter represents the radix or base to be used and it is optional.

Return value: It returns a number and if the first character can't be converted to a number then the function returns NaN. It actually returns a number parsed up to that point where it encounters a character which is not a number in the specified radix(base).

Example:

Input: `var n = parseInt("2018@geeksforgeeks");`

Output: `n = 2018`

Number() Function

Number() is an in built javascript function which is used to convert data type to number.

Syntax:

Number(query javascript variable)

Parameters: The parameter of the number function is the javascript variable which is user input to be converted. The number function converts any type of javascript variable to number type.

Return Values:

Number() function returns the number format for any type of javascript variable.

Examples:

Input : `Number(true);`

`Number(false);`

Output :

1

0

JAVASCRIPT – EVENTS

What is an Event ?

JavaScript's interaction with HTML is handled through events that occur when the user or the browser manipulates a page.

When the page loads, it is called an event. When the user clicks a button, that click too is an event. Other examples include events like pressing any key, closing a window, resizing a window, etc.

ONCLICK EVENT TYPE

This is the most frequently used event type which occurs when a user clicks the left button of his mouse. You can put your validation, warning etc., against this event type.

Example

```
<html>
<head>
  <script type = "text/javascript">
    <!--
      function sayHello() {
        alert("Hello World")
      }
    //-->
  </script>
</head>

<body>
  <p>Click the following button and see result</p>
  <form>
    <input type = "button" onclick = "sayHello()" value = "Say Hello" />
  </form>
</body>
</html>
```

ONSUBMIT EVENT TYPE

onsubmit is an event that occurs when you try to submit a form. You can put your form validation against this event type.

Example

The following example shows how to use onsubmit. Here we are calling a **validate()** function before submitting a form data to the webserver. If **validate()** function returns true, the form will be submitted, otherwise it will not submit the data.

Try the following example.

```
<html>
<head>
  <script type = "text/javascript">
    <!--
      function validation() {
        all validation goes here
        .....
        return either true or false
      }
    //-->
  </script>
</head>

<body>
  <form method = "POST" action = "t.cgi" onsubmit = "return validate()">
    .....
    <input type = "submit" value = "Submit" />
  </form>
</body>
</html>
```

HTML 5 STANDARD EVENTS

The standard HTML 5 events are listed here for your reference. Here script indicates a Javascript function to be executed against that event.

Attribute	Value	Description
Offline	script	Triggers when the document goes offline
onblur	script	Triggers when the window loses focus
oncanplay	script	Triggers when media can start play, but might has to stop for buffering
oncanplaythrough	script	Triggers when media can be played to the end, without stopping for buffering
onchange	script	Triggers when an element changes
onclick	script	Triggers on a mouse click
ondrag	script	Triggers when an element is dragged
ondrop	script	Triggers when dragged element is being dropped
ondurationchange	script	Triggers when the length of the media is changed
onemptied	script	Triggers when a media resource element suddenly becomes empty.
onended	script	Triggers when media has reach the end
onerror	script	Triggers when an error occur
onfocus	script	Triggers when the window gets focus

onkeydown	script	Triggers when a key is pressed
onkeypress	script	Triggers when a key is pressed and released
onkeyup	script	Triggers when a key is released
onload	script	Triggers when the document loads
onmousedown	script	Triggers when a mouse button is pressed
onmousemove	script	Triggers when the mouse pointer moves
onmouseout	script	Triggers when the mouse pointer moves out of an element
onmouseover	script	Triggers when the mouse pointer moves over an element
onmouseup	script	Triggers when a mouse button is released
onprogress	script	Triggers when the browser is fetching the media data
onratechange	script	Triggers when the media data's playing rate has changed
onreadystatechange	script	Triggers when the ready-state changes
onredo	script	Triggers when the document performs a redo
onresize	script	Triggers when the window is resized
onscroll	script	Triggers when an element's scrollbar is being scrolled
onsubmit	script	Triggers when a form is submitted

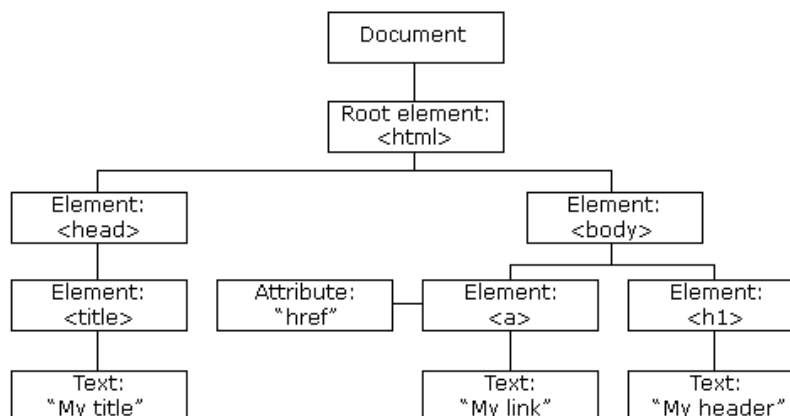
JAVASCRIPT - DOCUMENT OBJECT MODEL OR DOM

The way a document content is accessed and modified is called the **Document Object Model**, or **DOM**. The Objects are organized in a hierarchy. This hierarchical structure applies to the organization of objects in a Web document.

- **Window object** – Top of the hierarchy. It is the outmost element of the object hierarchy.
- **Document object** – Each HTML document that gets loaded into a window becomes a document object. The document contains the contents of the page.
- **Form object** – Everything enclosed in the `<form>...</form>` tags sets the form object.
- **Form control elements** – The form object contains all the elements defined for that object such as text fields, buttons, radio buttons, and checkboxes.

The **HTML DOM** model is constructed as a tree of **Objects**:

The HTML DOM Tree of Objects



Example:

`<Table>`

`<ROWS>`

`<TR>`

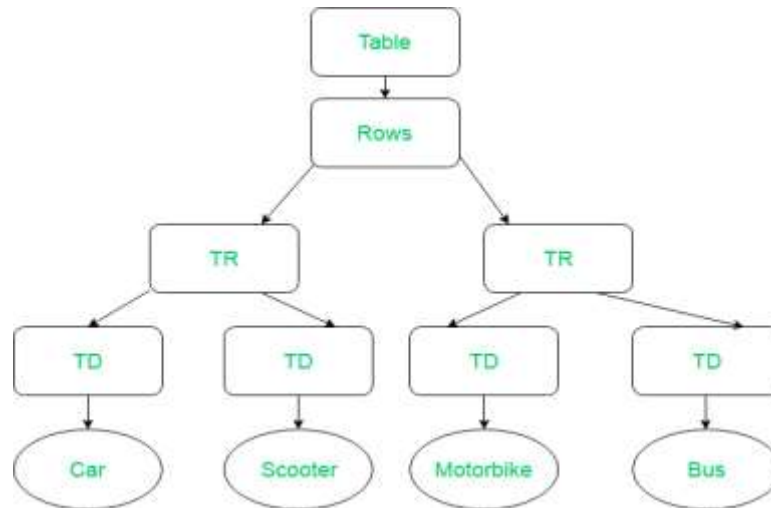
`<TD>Car</TD>`

`<TD>Scooter</TD>`

```

</TR>
<TR>
  <TD>MotorBike</TD>
  <TD>Bus</TD>
</TR>
</ROWS>
</Table>

```



HTML DOM getElementById() Method

Example

Get the element with the specified ID:

```
document.getElementById("demo");
```

Definition and Usage

The `getElementById()` method returns the element that has the ID attribute with the specified value.

This method is one of the most common methods in the HTML DOM, and is used almost every time you want to manipulate, or get info from, an element on your document.

Returns *null* if no elements with the specified ID exists.

An ID should be unique within a page. However, if more than one element with the specified ID exists, the `getElementById()` method returns the first element in the source code.

HTML DOM getElementsByName() Method

Example

Get all elements with the specified name:

```
var x = document.getElementsByName("fname");
```

Definition and Usage

The `getElementsByName()` method returns a collection of all elements in the document with the specified name (the **value** of the name attribute), as a `NodeList` object.

The `NodeList` object represents a collection of nodes. The nodes can be accessed by index numbers. The index starts at 0.

JAVASCRIPT OBJECTS

Objects, in JavaScript, is it's most important data-type and forms the building blocks for modern JavaScript. These objects are quite different from JavaScript's primitive data-types (Number, String, Boolean, null, undefined and symbol) in the sense that while these primitive data-types all store a single value each (depending on their types).

- Objects are more complex and each object may contain any combination of these primitive data-types as well as reference data-types.
- An object, is a reference data type. Variables that are assigned a reference value are given a reference or a pointer to that value. That reference or pointer points to the location in memory where the object is stored. The variables don't actually store the value.
- Loosely speaking, **objects in JavaScript may be defined as an unordered collection of related data, of primitive or reference types, in the form of "key: value" pairs.** These keys can be variables or functions and are called properties and methods, respectively, in the context of an object.

Example

```
var person = {firstName:"John", lastName:"Doe", age:50, eyeColor:"blue"};
```

Object Properties

The named values, in JavaScript objects, are called **properties**.

Property	Value
----------	-------

firstName	John
lastName	Doe
age	50
eyeColor	blue

Using the JavaScript Keyword new

The following example also creates a new JavaScript object with four properties:

Example

```
var person = new Object();  
person.firstName = "John";  
person.lastName = "Doe";  
person.age = 50;  
person.eyeColor = "blue";
```
